

Micro820 MSG_MODBUS to Automation Direct GS10/GS11 VFD

Step-by-Step Implementation Guide for Modbus RTU Communication

Based on the methodology shown in *Micro800 Modbus RTU Serial Configuration to a PowerFlex Drive* (YouTube), adapted for Automation Direct DURApulse GS10/GS11 drives with Allen-Bradley Micro820 PLC.

This guide walks through every parameter, every register address, and every MSG_MODBUS configuration step needed to get a Micro820 controlling a GS10 or GS11 VFD over RS-485 Modbus RTU. The approach mirrors the PowerFlex 525 video tutorial but translates all registers and parameters to the Automation Direct equivalents.

Hardware	Allen-Bradley Micro820 (2080-LC20-20QWB) + Automation Direct GS10 or GS11
Software	Connected Components Workbench (CCW) v12+
Protocol	Modbus RTU over RS-485 (2-wire)
PLC Role	Modbus RTU Master
VFD Role	Modbus RTU Slave
Date	May 2026
Author	FactoryLM / MIRA

Table of Contents

1. How the PowerFlex Video Works (and How We Adapt It)
 2. Hardware Wiring: Micro820 RS-485 to GS10/GS11 RJ45
 3. GS10/GS11 VFD Parameter Setup
 4. GS10/GS11 Modbus Register Map (vs PowerFlex 525)
 5. CCW Serial Port Configuration (Micro820)
 6. MSG_MODBUS Instruction Setup in CCW
 7. Ladder Logic: Write Speed + Run/Stop Commands
 8. Ladder Logic: Read Status + Frequency Feedback
 9. Block Transfer Optimization (Fewer Modbus Messages)
 10. Testing & Commissioning Checklist
 11. Troubleshooting Common Failures
- Appendix A: Complete GS10 Communication Parameters (P09.xx)
- Appendix B: Complete GS10 Status Address Map
- Appendix C: PowerFlex 525 vs GS10 Register Cross-Reference

1. How the PowerFlex Video Works (and How We Adapt It)

The YouTube video *Micro800 Modbus RTU Serial Configuration to a PowerFlex Drive* demonstrates a straightforward approach:

Step 1: Configure the PowerFlex 525 parameters for Modbus RTU communication (P046 Start Source = Serial/DSI, P047 Speed Reference = Serial/DSI, baud rate, node address).

Step 2: Configure the Micro820 serial port in CCW as Modbus RTU Master.

Step 3: Add MSG_MODBUS instructions in ladder logic to write the Logic Command word (register 0x2000 = 8192) and Speed Reference (register 0x2001 = 8193).

Step 4: Add MSG_MODBUS instructions to read Logic Status (0x2100 = 8448) and Output Frequency (0x2103 = 8451) back from the drive.

Step 5: Chain MSG_MODBUS instructions using the .DN (Done) bit so they execute sequentially.

The good news: **the Automation Direct GS10/GS11 uses the exact same Modbus register addresses for control and status.** Register 0x2000 is the Control Word, 0x2001 is the Frequency Command, and 0x2100-0x210C are the status monitors. The only differences are in the VFD-side parameter numbers and the bit definitions within the control/status words.

Key Differences: PowerFlex 525 vs GS10/GS11

Concept	PowerFlex 525	GS10 / GS11
Start Source param	P046 = 3 (Serial/DSI)	P00.21 = 02 (RS-485 comm)
Speed Reference param	P047 = 3 (Serial/DSI)	P00.20 = 1 (RS-485 comm)
Comm protocol param	P038/P039	P09.04 (1-17, see Section 3)
Node address param	P036	P09.00 (1-254)
Baud rate param	P038	P09.01 (4.8-38.4 Kbps)
Control Word register	0x2000 (8192)	0x2000 (8192) - SAME
Freq Command register	0x2001 (8193)	0x2001 (8193) - SAME
Status register	0x2100 (8448)	0x2100 (8448) - SAME
Physical connection	DSI RJ45 or screw terminals	RS-485 RJ45 port
MSG instruction	MSG_MODBUS	MSG_MODBUS - SAME

2. Hardware Wiring: Micro820 RS-485 to GS10/GS11 RJ45

GS10/GS11 RJ45 Serial Port Pinout

Pin	Signal	Description
1	Reserved	Do not connect
2	Reserved	Do not connect
3	SGND	Signal Ground
4	SG-	RS-485 Data- (B line)
5	SG+	RS-485 Data+ (A line)
6	Reserved	Do not connect
7	Reserved	Do not connect
8	+10VS	+10V supply (do NOT connect)

Micro820 Embedded RS-485 Serial Port (Screw Terminals)

The Micro820 has a combo RS-232/RS-485 port on screw terminals. For RS-485 mode, use:

Terminal	Signal	Connect To
Pin 1 (TX+/RX+)	RS-485 A (Data+)	GS10 RJ45 Pin 5 (SG+)
Pin 2 (TX-/RX-)	RS-485 B (Data-)	GS10 RJ45 Pin 4 (SG-)
Pin 5 (GND)	Signal Ground	GS10 RJ45 Pin 3 (SGND)

WARNING: If using a USB-to-RS485 adapter instead of the Micro820 embedded port, the adapter typically provides fail-safe bias resistors (680 ohm pullup on A, pulldown on B). The Micro820 embedded port and the GS10 may NOT provide adequate bias. Add external 680 ohm bias resistors if the bus idles with 0mV between A and B.

Termination Resistors

Install a 120 ohm termination resistor across SG+ and SG- at the GS10 end (required per the GS10 manual). If the Micro820 is at the other end of the RS-485 bus, add a second 120 ohm termination resistor at that end as well. For short cable runs under 3 meters, you may get away without termination, but it is best practice.

Use shielded twisted pair cable (Belden 9842 or AutomationDirect L19954 recommended). Connect shield to SGND at one end only.

TIP: A standard Ethernet patch cable (straight-through, NOT crossover) can be used for GS10-to-GS10 daisy chains. For PLC-to-GS10, you will likely need to make a custom cable or use a breakout adapter from the RJ45 to screw terminals.

3. GS10/GS11 VFD Parameter Setup

Before the Micro820 can communicate with the drive, you must configure the VFD parameters through the keypad or GSoft2 software. Set these parameters in the following order:

Step 3.1: Set Operation Source to RS-485

Parameter	Set To	Description
P00.20	1	1st Source of Frequency Command [Remote] = RS-485 communication
P00.21	02	1st Source of Operation Command [Remote] = RS-485 communication
P00.30	1	2nd Source of Frequency Command [Local] = RS-485 communication
P00.31	02	2nd Source of Operation Command [Local] = RS-485 comm (keypad STOP enabled via P00.32)

TIP: Setting P00.31 to 02 keeps the keypad STOP button active as a safety override. If you set P00.31 to 03, the keypad STOP is disabled and the drive can only be stopped via RS-485. Keep it at 02 during commissioning.

Step 3.2: (Optional) Set a Multi-Function Input for Local/Remote

If you want a physical switch to toggle between local keypad control and remote RS-485 control, set one of the digital inputs to function 56 (Local/Remote selection):

P02.01 through P02.05 (DI1-DI5) = 56

Step 3.3: Set Communication Parameters

Parameter	Set To	Description
P09.00	1	Communication Address (1-254). Use 1 for single drive.
P09.01	19.2	Transmission Speed = 19200 baud (must match Micro820)
P09.04	14	COM1 Protocol = 8 data bits, Even parity, 1 stop bit (RTU)

WARNING: P09.04 is the most common source of communication failure. The Micro820 CCW default serial port configuration is 19200 baud, 8 data bits, Even parity, 1 stop bit. That corresponds to P09.04 = **14** on the GS10/GS11. If you set the GS10 to P09.04 = 13 (8-N-2) but the Micro820 is configured for Even parity, you will get framing errors and every byte will read as 0x3F. This exact mismatch has caused hours of debugging on real installations.

GS10/GS11 P09.04 Protocol Options (RTU modes only)

P09.04 Value	Data Bits	Parity	Stop Bits	Mode
12	8	None	1	RTU
13 (default)	8	None	2	RTU
14	8	Even	1	RTU
15	8	Odd	1	RTU
16	8	Even	2	RTU
17	8	Odd	2	RTU

Step 3.4: Set Communication Fault Handling (Optional but Recommended)

Parameter	Set To	Description
P09.02	1 or 2	COM1 fault treatment: 1=Fault+ramp to stop, 2=Fault+coast to stop, 3=No fault (default)
P09.03	5.0	COM1 timeout detection: 5.0 seconds. If no Modbus message received in 5s, trigger fault action.
P09.09	2.0	Response delay: 2.0 ms (default). Increase if PLC has slow turnaround.

4. GS10/GS11 Modbus Register Map

These are the registers you will read from and write to using MSG_MODBUS instructions in the Micro820. The addressing is 0-based (hex). Some Modbus masters use 1-based addressing, in which case add 1 to the decimal value.

Control Registers (Write)

Register (Hex)	Register (Dec)	Name	Access	Description
0x2000	8192	Control Word 1	Write	Run/Stop, Direction, Jog, Accel/Decel select, Multi-speed select
0x2001	8193	Control Word 2 (Freq Cmd)	Write	Frequency command in 0.01 Hz units (e.g., 6000 = 60.00 Hz)
0x2002	8194	Control Word 3	Write	External Fault, Reset, Base Block, Fire Mode

Control Word 1 (0x2000) Bit Definitions

Bits	Value (Binary)	Meaning
Bits 1-0	00	No function
Bits 1-0	01	STOP
Bits 1-0	10	RUN
Bits 1-0	11	JOG + RUN
Bits 5-4	00	No function
Bits 5-4	01	Forward (FWD)
Bits 5-4	10	Reverse (REV)
Bits 5-4	11	Change direction
Bits 7-6	00	1st accel/decel time
Bits 7-6	01	2nd accel/decel time
Bits 7-6	10	3rd accel/decel time
Bits 7-6	11	4th accel/decel time
Bit 12	1	Enable bits 6-11 (multi-speed select)

Common Control Word Values (Quick Reference)

Action	Decimal Value	Hex	Binary (key bits)
Stop	1	0x0001	Bits 1:0 = 01
Run Forward	18	0x0012	Bits 1:0 = 10 (Run), Bits 5:4 = 01 (FWD)
Run Reverse	34	0x0022	Bits 1:0 = 10 (Run), Bits 5:4 = 10 (REV)
Jog Forward	19	0x0013	Bits 1:0 = 11 (Jog+Run), Bits 5:4 = 01 (FWD)

Status Registers (Read)

Register (Hex)	Register (Dec)	Name	Description
0x2100	8448	Status Monitor 1	Error/Fault code (see Appendix B)
0x2101	8449	Status Monitor 2	Drive status bits (running, direction, control source)
0x2102	8450	Frequency Command	Commanded frequency (XXX.XX Hz)
0x2103	8451	Output Frequency	Actual output frequency (XXX.XX Hz)
0x2104	8452	Output Current	Drive output current (XX.XX A)
0x2105	8453	DC Bus Voltage	DC bus voltage (XXX.X V)
0x2106	8454	Output Voltage	Output voltage (XXX.X V)
0x2107	8455	Multi-speed Step	Current multi-speed step number
0x210B	8459	Output Torque	Output torque (XXX.X %)
0x210C	8460	Actual Motor Speed	Motor speed (XXXXX RPM)

5. CCW Serial Port Configuration (Micro820)

In Connected Components Workbench, configure the Micro820 serial port to act as a Modbus RTU Master. This is the PLC-side equivalent of the drive parameter setup.

Step 5.1: Open Serial Port Configuration

In CCW, expand your Micro820 project tree. Navigate to **Controller Settings** then **Serial Port**. Select the embedded serial port (Port 0).

Step 5.2: Set Protocol and Parameters

CCW Setting	Value	Must Match GS10 Parameter
Protocol	Modbus RTU	N/A (both must be RTU)
Mode	Master	N/A (GS10 is always Slave)
Physical Layer	RS-485	N/A (GS10 is RS-485)
Baud Rate	19200	P09.01 = 19.2
Data Bits	8	P09.04 bits (8 for all RTU)
Parity	Even	P09.04 = 14 (8, E, 1)
Stop Bits	1	P09.04 = 14 (8, E, 1)

WARNING: The baud rate, data bits, parity, and stop bits **MUST** match between the Micro820 and the GS10/GS11. A single mismatch will cause communication failure. The most reliable combination is 19200 / 8 / Even / 1 (P09.04 = 14 on the GS10).

Step 5.3: Download Configuration

After setting the serial port parameters, download the project to the Micro820. The serial port configuration takes effect after the download completes. You may need to power-cycle the PLC for the serial port changes to apply.

6. MSG_MODBUS Instruction Setup in CCW

The MSG_MODBUS instruction is the Micro820's built-in function block for Modbus RTU communication. Each MSG_MODBUS instance handles one Modbus transaction (read or write). You will need at minimum 2 instances: one to write commands to the drive and one to read status back.

MSG_MODBUS Parameters

Parameter	Type	Description
EN (Enable)	BOOL input	Rising edge or level triggers the message. Use .DN from previous MSG to chain.
Channel	INT	Serial port number. 0 = embedded port on Micro820.
SlaveAddr	INT	Modbus slave address of the GS10 (matches P09.00, typically 1).
FunctionCode	INT	3 = Read Holding Registers, 6 = Write Single Register, 16 = Write Multiple Registers.
DataAddr	INT	Starting register address (0-based). For GS10 control: 8192 (0x2000).
DataLen	INT	Number of registers to read or write.
DataPtr	INT array	Pointer to tag array for data exchange (read results or write source).
DN (Done)	BOOL output	Set TRUE when transaction completes successfully.
ER (Error)	BOOL output	Set TRUE if transaction fails. Check ErrorID for code.
ErrorID	INT output	Error code: 0x0001=Timeout, 0x0002=CRC error, etc.

Critical: Trigger Type

The MSG_MODBUS instruction executes asynchronously at the end of each ladder scan. Do NOT leave the Enable input permanently HIGH. Instead, use a TON timer (e.g., 500ms-1000ms preset) to pulse the Enable, or chain messages using the .DN bit from the previous message. A permanently-enabled MSG_MODBUS can flood the serial bus and cause timeouts.

TIP: The Micro820 processes up to 4 MSG_MODBUS requests per scan per channel. If you have more than 4 messages, distribute them across multiple scans using a state machine or TON-based sequencing.

7. Ladder Logic: Write Speed + Run/Stop Commands

This section shows the ladder logic structure for writing the Control Word and Frequency Command to the GS10 drive. You need two MSG_MODBUS instances chained together.

Tags to Create in CCW

Tag Name	Type	Description
WriteControlWord	INT	Value to write to Control Word 1 (0x2000). See bit table in Section 4.
WriteFreqCmd	INT	Frequency command in 0.01 Hz units. 6000 = 60.00 Hz, 3000 = 30.00 Hz.
MSG_Write_Ctrl	MSG_MODBUS	MSG instance for Control Word write.
MSG_Write_Freq	MSG_MODBUS	MSG instance for Frequency Command write.
WriteTrigger	BOOL	Pulse to initiate write sequence.
WriteTimer	TON	Timer to auto-repeat writes (e.g., every 500ms).

Rung 1: Write Timer (Auto-Repeat)

A TON timer generates periodic pulses to trigger the write sequence. When the timer reaches its preset (500ms), it resets and triggers the first write.

```
TON(WriteTimer, T#500ms)
WriteTimer.DN triggers WriteTrigger
```

Rung 2: Write Control Word (MSG_MODBUS #1)

Configure MSG_Write_Ctrl:

Parameter	Value	Explanation
EN	WriteTrigger (rising edge)	Triggered by timer
Channel	0	Embedded serial port
SlaveAddr	1	GS10 address (P09.00)
FunctionCode	6	Write Single Register
DataAddr	8192	0x2000 = Control Word 1

Parameter	Value	Explanation
DataLen	1	One register
DataPtr	WriteControlWord	Tag containing the value to write

To run forward at 60 Hz, set WriteControlWord = 18 (decimal) and WriteFreqCmd = 6000.

To stop, set WriteControlWord = 1 (decimal).

Rung 3: Write Frequency Command (MSG_MODBUS #2)

Configure MSG_Write_Freq:

Parameter	Value	Explanation
EN	MSG_Write_Ctrl.DN	Chained: starts after Control Word write completes
Channel	0	Same serial port
SlaveAddr	1	Same drive
FunctionCode	6	Write Single Register
DataAddr	8193	0x2001 = Frequency Command
DataLen	1	One register
DataPtr	WriteFreqCmd	Tag containing frequency in 0.01 Hz units

TIP: Alternative: Use Function Code 16 (Write Multiple Registers) to write both the Control Word and Frequency Command in a single message. Set DataAddr = 8192, DataLen = 2, and point DataPtr to a 2-element INT array where [0] = Control Word and [1] = Frequency Command. This halves your write messages.

8. Ladder Logic: Read Status + Frequency Feedback

Tags to Create

Tag Name	Type	Description
StatusData	INT[6]	Array to receive status registers (0x2100-0x2105).
MSG_Read_Status	MSG_MODBUS	MSG instance for status read.
DriveErrorCode	INT	= StatusData[0] (Status Monitor 1 - fault code)
DriveStatusBits	INT	= StatusData[1] (Status Monitor 2 - run/stop/dir)
FreqCommand	INT	= StatusData[2] (Commanded frequency)
OutputFrequency	INT	= StatusData[3] (Actual output frequency)
OutputCurrent	INT	= StatusData[4] (Output current)
DCBusVoltage	INT	= StatusData[5] (DC bus voltage)

Rung 4: Read Status (MSG_MODBUS #3)

Configure MSG_Read_Status:

Parameter	Value	Explanation
EN	MSG_Write_Freq.DN	Chained: starts after frequency write completes
Channel	0	Embedded serial port
SlaveAddr	1	GS10 address
FunctionCode	3	Read Holding Registers
DataAddr	8448	0x2100 = Status Monitor 1 (starting address)
DataLen	6	Read 6 consecutive registers (0x2100-0x2105)
DataPtr	StatusData	INT[6] array receives the response

Rung 5: Map Status Data to Named Tags

After MSG_Read_Status.DN goes TRUE, copy the array elements to named tags for readability. Use MOV instructions or direct assignment in Structured Text:

```
DriveErrorCode := StatusData[0]; // Fault code (0 = no fault)
DriveStatusBits := StatusData[1]; // Run/Stop/Direction bits
FreqCommand := StatusData[2]; // Commanded freq (x 0.01 Hz)
OutputFrequency := StatusData[3]; // Actual freq (x 0.01 Hz)
OutputCurrent := StatusData[4]; // Current (x 0.01 A)
DCBusVoltage := StatusData[5]; // DC bus (x 0.1 V)
```

Complete Message Chain Summary

The full cycle repeats every ~500ms (or whatever your WriteTimer preset is):

```
Timer.DN --> MSG_Write_Ctrl (FC6, 0x2000) --> .DN -->
MSG_Write_Freq (FC6, 0x2001) --> .DN -->
MSG_Read_Status (FC3, 0x2100, 6 regs) --> .DN -->
Timer resets, cycle repeats
```

9. Block Transfer Optimization (Fewer Modbus Messages)

The GS10/GS11 has a powerful Block Transfer feature (P09.11 through P09.26) that lets you consolidate up to 16 different parameters from scattered register groups into one contiguous block. This means you can read or write all your critical data with a single Modbus message instead of 6 separate ones.

How Block Transfer Works

Each Block Transfer parameter (P09.11 through P09.26) is a pointer that you configure to point at any other register in the drive. When you read P09.11 via Modbus, the drive returns the current value of whatever P09.11 points to. When you write to P09.11 via Modbus, the value gets written to the pointed-to register.

Example: Consolidate Everything into 1 Read + 1 Write

Block Transfer	GS10 Param	Set Value To	Points To	Access
P09.11	Block 1	0 (0x0000)	P00.00 Drive ID	Read
P09.12	Block 2	6 (0x0006)	Firmware version	Read
P09.13	Block 3	8448 (0x2100)	Status Monitor 1	Read
P09.14	Block 4	8449 (0x2101)	Status Monitor 2	Read
P09.15	Block 5	8451 (0x2103)	Output Frequency	Read
P09.16	Block 6	268 (0x010C)	Accel Time 1 (P01.12)	Write
P09.17	Block 7	269 (0x010D)	Decel Time 1 (P01.13)	Write
P09.18	Block 8	8192 (0x2000)	Control Word 1	Write
P09.19	Block 9	8193 (0x2001)	Freq Command	Write

With this setup, one Modbus read of P09.11 through P09.15 (Modbus address 42316, length 5) gets you drive ID, firmware, fault status, running status, and output frequency. One Modbus write to P09.16 through P09.19 (Modbus address 42321, length 4) sets accel time, decel time, control word, and frequency command. That is 2 messages instead of the 6+ you would normally need.

Block Transfer Pointer Value Calculation

For drive parameters (P-group), the pointer value is calculated as: Group number (hex, upper byte) + Parameter number (hex, lower byte), converted to decimal. Example: P02.22 = 0x0200 + 0x16 = 0x0216 = 534 decimal. For status registers (0x2000, 0x2100 range), use the hex address directly converted to decimal (e.g., 0x2000 = 8192, 0x2100 = 8448).

10. Testing & Commissioning Checklist

Before Power-Up

- ☐ Verify RS-485 wiring: Micro820 Pin 1 (A+) to GS10 RJ45 Pin 5 (SG+)
- ☐ Verify RS-485 wiring: Micro820 Pin 2 (B-) to GS10 RJ45 Pin 4 (SG-)
- ☐ Verify Ground: Micro820 Pin 5 (GND) to GS10 RJ45 Pin 3 (SGND)
- ☐ Termination resistor installed at GS10 end (120 ohm across SG+ and SG-)
- ☐ Shield connected to ground at ONE end only

GS10/GS11 VFD Parameters Verified

- ☐ P00.20 = 1 (Freq source = RS-485)
- ☐ P00.21 = 02 (Operation source = RS-485)
- ☐ P09.00 = 1 (or your chosen address)
- ☐ P09.01 = 19.2 (19200 baud)
- ☐ P09.04 = 14 (8, Even, 1 stop, RTU) -- MUST match Micro820

CCW / Micro820 Configuration Verified

- ☐ Serial port = Modbus RTU Master, RS-485
- ☐ Baud = 19200, Parity = Even, Stop bits = 1
- ☐ Project downloaded to Micro820

First Communication Test

- ☐ Set WriteControlWord = 0 (no command), WriteFreqCmd = 0
- ☐ Go online in CCW, monitor MSG_Read_Status.DN and .ER bits
- ☐ Confirm MSG_Read_Status.DN pulses TRUE (communication working)
- ☐ Confirm StatusData[0] = 0 (no faults)
- ☐ If MSG_Read_Status.ER = TRUE, check ErrorID and see Section 11

First Motor Test (DISCONNECT LOAD FIRST)

- ☐ Set WriteFreqCmd = 500 (5.00 Hz - slow speed for testing)

- ☐ Set WriteControlWord = 18 (Run Forward)
- ☐ Verify motor spins forward slowly
- ☐ Set WriteControlWord = 1 (Stop)
- ☐ Verify motor stops
- ☐ Set WriteControlWord = 34 (Run Reverse)
- ☐ Verify motor spins reverse
- ☐ Set WriteControlWord = 1 (Stop)

Full Speed Test

- ☐ Set WriteFreqCmd = 6000 (60.00 Hz)
- ☐ Set WriteControlWord = 18 (Run Forward)
- ☐ Confirm OutputFrequency (StatusData[3]) reads back ~6000
- ☐ Confirm OutputCurrent (StatusData[4]) is reasonable for motor
- ☐ Set WriteControlWord = 1 (Stop)

11. Troubleshooting Common Failures

Symptom	Likely Cause	Fix
MSG_MODBUS.ER = TRUE, ErrorID = 0x0001 (Timeout)	No response from drive. Wiring, address, or baud mismatch.	1) Check wiring (A/B not swapped). 2) Verify P09.00 matches SlaveAddr. 3) Verify baud matches.
MSG_MODBUS.ER = TRUE, ErrorID = 0x0002 (CRC Error)	Parity or stop bit mismatch between PLC and drive.	Verify P09.04 on GS10 matches CCW parity/stop settings. Most common: GS10 default is 13 (8-N-2) but CCW default is Even parity.
Every byte reads as 0x3F	Framing error due to parity/stop bit mismatch.	Set P09.04 = 14 (8-E-1) to match CCW default of Even parity, 1 stop bit.
Drive does not start when WriteControlWord = 18	Operation source not set to RS-485, or drive in Local mode.	Verify P00.21 = 02. Check if a DI is set to Local/Remote (function 56) and the input is forcing Local mode.
Drive runs but ignores frequency command	Frequency source not set to RS-485.	Verify P00.20 = 1. If in Local mode, also check P00.30 = 1.
Communication works initially then drive faults (F081 or CE10)	Communication timeout. PLC stopped sending messages.	Increase P09.03 (timeout detection) or ensure PLC message cycle is faster than timeout.
0mV between SG+ and SG- when bus is idle	Missing bias resistors. Bus floats to indeterminate state.	Add 680 ohm pullup to +5V on SG+ and 680 ohm pulldown to GND on SG-. Or use a USB-RS485 adapter with built-in bias as your network master.
StatusData values are all zero or garbage	Wrong DataAddr in MSG_Read_Status.	Verify DataAddr = 8448 (0x2100), not 8192. 8192 is the CONTROL word, 8448 is the STATUS word.

Appendix A: Complete GS10 Communication Parameters (P09.xx)

Parameter	Modbus Addr	Description	Range	Default
P09.00	42305	Communication address	1-254	1
P09.01	42306	Transmission speed (Kbps)	4.8-38.4	38.4
P09.02	42307	COM1 fault treatment	0-3	3 (no fault)
P09.03	42308	COM1 timeout detection (sec)	0.0-100.0	0.0 (off)
P09.04	42309	COM1 protocol (data/parity/stop)	1-17	13 (8-N-2 RTU)
P09.09	42314	Response delay (ms)	0.0-200.0	2.0
P09.10	42315	Comm main frequency (Hz)	0.00-599.00	60.00
P09.11-P09.26	42316-42331	Block Transfer 1-16	0-65535	0
P09.30	42335	Comm decoding method	0-1	1

Appendix C: PowerFlex 525 vs GS10 Register Cross-Reference

This is the direct translation table. If you are following a PowerFlex 525 Modbus tutorial and need to know the GS10 equivalent, use this table.

Function	PF525 Register	PF525 Param	GS10 Register	GS10 Param	Notes
Logic Command	0x2000 (8192)	P046=3	0x2000 (8192)	P00.21=02	SAME register, different setup param
Speed Reference	0x2001 (8193)	P047=3	0x2001 (8193)	P00.20=1	SAME register, 0.01 Hz units on both
Node Address	P036	-	-	P09.00	Different param number
Baud Rate	P038	-	-	P09.01	Different param number
Data Format	P039	-	-	P09.04	Different encoding scheme
Status/Faults	0x2100 (8448)	-	0x2100 (8448)	-	SAME register, different bit defs

Function	PF525 Register	PF525 Param	GS10 Register	GS10 Param	Notes
Output Frequency	0x2103 (8451)	-	0x2103 (8451)	-	SAME register
Output Current	0x2104 (8452)	-	0x2104 (8452)	-	SAME register
DC Bus Voltage	0x2105 (8453)	-	0x2105 (8453)	-	SAME register

The key takeaway: the Modbus register addresses for control (0x2000-0x2002) and status (0x2100-0x210C) are identical between PowerFlex 525 and GS10/GS11. The differences are only in the VFD-side setup parameters and the bit definitions within the control/status words. Any PowerFlex 525 Modbus RTU tutorial can be directly applied to GS10/GS11 with the parameter translations in this guide.